The Problem

Widgets are normally embedded on websites the platform owner doesn't control, which means the public submission endpoint receives requests from arbitrary, untrusted origins at unpredictable volume. This project builds a signup-widget platform around that constraint: a tenant creates a widget through an authenticated API, embeds it anywhere via a single script tag, and the backend must safely validate, rate-limit, enrich, and store whatever the public internet sends back — without ever trusting the client or letting a failing dependency take the system down.

Data Model

The system has three tables, each mapping directly to one of the three request paths described in Section 1: tenants own widgets, widgets receive submissions, and every submission carries its tenant forward for isolation.

Tenants

Represents each customer/account on the platform. Fields are shaped to sit on top of Supabase Auth rather than duplicate it — this table stores account-level data, not credentials.

Field	Type	Constraints / Purpose
`id`	UUID	Primary key; matches `auth.users.id`
`email`	TEXT	Mirrored from Supabase Auth for display/dashboard use
`company_name`	TEXT	Nullable; displayed in dashboard
`created_at`	TIMESTAMPTZ	Creation timestamp

`updated_at`	TIMESTAMPTZ	Last update timestamp

Widgets

Represents each signup-form widget a tenant creates. Scoped to type = "signup" for this build — the column stays a string rather than an enum so a second type can be added later without a migration, but only "signup" is implemented now.

Field	Type	Constraints / Purpose
`id`	UUID	Primary key; used by the embed snippet
`tenant_id`	UUID	FK → `tenants.id`; used for tenant isolation
`type`	TEXT	Fixed to `signup` for this build
`title`	TEXT	Widget heading
`description`	TEXT	Nullable; widget subtext
`form_fields`	JSONB	Ordered array of field definitions
`button_text`	TEXT	Defaults to `Sign up`
`display_options`	JSONB	Position, trigger, and theming

`created_at`	TIMESTAMPTZ	Creation timestamp
`updated_at`	TIMESTAMPTZ	Last update timestamp

Index: idx_widgets_tenant_id on tenant_id — every widget-management query is filtered by the authenticated tenant, so this index is load-bearing for the isolation guarantee, not just performance.

form_fields shape — an ordered array the frontend renders directly, and the backend validates submissions against field-by-field:

[
{ "name": "email", "label": "Email address", "type": "email", "required": true },
{ "name": "full_name", "label": "Full name", "type": "text", "required": false },
{ "name": "_hp_website", "label": "Website", "type": "text", "required": false, "honeypot": true
}
]

display_options shape — presentation only, never touched by validation logic:

{ "position": "bottom-right", "trigger": "on_load", "delay_seconds": 0, "theme_color":
"#4F46E5" }

Submissions

Represents each visitor's completed (or attempted) signup. tenant_id is denormalized onto this table rather than joined through widgets on every read, because the dashboard's most common query pattern is "all submissions for this tenant" — joining through widgets for every dashboard page load would be wasteful when the FK can just be copied at insert time.

Field	Type	Constraints / Purpose
`id`	UUID	Primary key

`widget_id`	UUID	Foreign key → `widgets.id`
`tenant_id`	UUID	Foreign key → `tenants.id`; denormalized for direct tenant-scoped queries
`data`	JSONB	Submitted field values, keyed to `form_fields[].name`
`ip_address`	INET	Nullable; source for geo enrichment; not exposed via API
`country`	TEXT	Nullable; populated by geo enrichment; null if all providers fail
`city`	TEXT	Nullable; populated by geo enrichment; null if all providers fail
`is_spam`	BOOLEAN	Default `false`; set true on honeypot hit; stored but excluded from dashboard counts by default
`created_at`	TIMESTAMPTZ	Submission timestamp

Indexes: idx_submissions_tenant_id and idx_submissions_widget_id — the former backs tenant isolation on the dashboard API, the latter backs per-widget stats (Requirement 6).

Tenant isolation enforcement: every query against widgets and submissions is scoped by tenant_id pulled from the authenticated session (Supabase JWT), never from a request parameter or body — a request can't claim a different tenant's ID and read their data, because the ID used for filtering is never client-supplied on authenticated routes.

The Embed Flow

The lifecycle has four steps, each crossing from one actor to the next.

1. Create — the tenant creates a signup widget via the authenticated Widget Management API. The response includes the widget's id and the ready-to-use embed snippet:

<script src="https://api.example.com/widget.js?id=abc123"></script>

2. Load — the tenant pastes that script tag into their own website (a separate origin from the API). When a visitor loads that page, the script fetches the widget's config from a public, cached endpoint using the id in the query string.
3. Render — the script uses the returned config (form_fields, display_options, button_text) to render the signup form directly on the customer's page — no additional assets or origins involved.
4. Submit — when the visitor fills out and submits the form, the script sends the data cross-origin to the public submission endpoint. The backend validates, checks the honeypot, rate-limits, enriches with geo data, stores the row, and fires a best-effort confirmation email — all before returning a response to the visitor's browser.

This is also the order the system is built in: Phase 2 hardens step 4 first since it is the highest-risk path, then Phase 3 wires up steps 2 and 3 around it.

API Surface / Contracts (for all three paths)

Endpoints are grouped by actor, matching the three paths in the architecture diagram. Auth is enforced via Supabase JWT; public endpoints take no auth but are still validated and rate-limited.

Widget owner (authenticated) — Widget Management API

POST /widgets — create a widget. Body: title, description, form_fields, button_text, display_options. Returns the created widget plus its embed snippet.
GET /widgets — list the authenticated tenant's widgets.
GET /widgets/:id — fetch one widget. 404 if it doesn't belong to the tenant.
PATCH /widgets/:id — update a widget. 404 if it doesn't belong to the tenant.
DELETE /widgets/:id — delete a widget. 404 if it doesn't belong to the tenant.

All queries scoped to the authenticated tenant_id, backed by the widget DB's tenant isolation (Section 2).

Customer website (any origin, public, cached, CORS)

GET /widget.js — the versioned JS loader script referenced by the embed snippet. Long-lived Cache-Control, cache-busted by URL on release.
GET /widgets/:id/config — public widget config (form_fields, display_options, button_text),

used to render the widget. Short-lived Cache-Control. Returns 404 for an unknown id, no tenant data leaked.

Website visitor (public, CORS)

OPTIONS /submissions — CORS preflight, handled explicitly.
POST /submissions — accepts { widget_id, data }. Pipeline, in order:

1. Validation — malformed or oversized payload → 4xx, never a 500.
2. Rate limit + spam check — flood or honeypot hit → 429 / dropped, service stays up.
3. Geo enrichment — Provider A, falls back to Provider B, falls back to storing without geo data.
4. Store submission.
5. Email/webhook side effect — failure here must not block the success response. Returns 201 on success.

Widget owner (authenticated) — Dashboard API

GET /widgets/:id/submissions — submissions for one widget, paginated.
GET /dashboard/stats — aggregate counts: total submissions, per-widget breakdown, geo breakdown, over time.